

Overall Impression

The orchestration layer is a solid, pragmatic shell-based wrapper around the more modern Python components (igos-build, pkm). It successfully stitches together a full LFS 13.0 + BLFS pipeline with resume capability, checkpointing, clean chroot management, and final image creation. The separation of concerns is good: master orchestrator → chroot lifecycle scripts → package tracking functions → tier-specific builders → image creator.

The design leans heavily on classic LFS shell practices while integrating the newer Python builder for the desktop tier. This hybrid approach works, but it introduces some friction and duplicated logic (especially around package tracking and CWD management).

1. Shell Scripting Safety

Good practices:

- set -euo pipefail in the master script (excellent).
- Many variables are quoted.
- Explicit cd "\$workdir" resets before each phase in build_ch8_package and similar functions — this directly addresses the CWD leakage problem you mentioned and is a good mitigation.
- trap for SIGINT/SIGTERM with a helpful resume message.

Remaining risks:

- Many tar, cp, rm -rf, chown -R, etc., still have unquoted expansions in some places (e.g., in phase_setup, sync_chroot_scripts, create-image.sh).
- pkg_manifest and file list generation use find ... | sed without strict quoting — safe in this context but fragile if filenames contain newlines (rare but possible).
- su - "\$BUILD_USER" -c "env -i ..." is correctly quoted but complex; any future change could introduce word-splitting.
- In chroot-enter.sh, the chroot ... /usr/bin/env -i ... \$CHROOT_CMD passes arguments through — generally safe because the command is controlled, but still relies on proper quoting inside the called scripts.
- pkg_remove and manifest parsing assume well-formed filenames (no newlines).

Verdict: Mostly safe for controlled build environment, but not bulletproof. The recent CWD reset helps a lot. Consider adding shopt -s nullglob and more consistent quoting. Use printf '%q' for complex command construction if needed.

2. Chroot Mount/Unmount Correctness

Strengths:

- chroot-setup.sh follows LFS 13.0 closely and uses mountpoint -q checks (idempotent).
- chroot-teardown.sh unmounts in reverse order and ignores failures (|| true).
- chroot-enter.sh verifies mounts before entering.

Issues:

- Teardown does **not** use -R (recursive) or lazy unmount (-l). If anything is busy (e.g., a shell left open, a background process), unmount can fail silently, leaving stale mounts on restart.
- /dev/shm handling is conditional but can leave remnants.
- No umount -R fallback or verification that nothing is still mounted under \$IGOS after teardown.
- save_checkpoint calls teardown/setup with > /dev/null 2>&1 || true — hides errors.

Risk of leaked mounts: Moderate. An interrupted build + manual restart can accumulate bind mounts. Recommend improving teardown with:

```
umount -R -l "$IGOS" 2>/dev/null || true
```

or a more robust loop that retries lazy unmounts.

3. DESTDIR Staging and Deployment (pkg-functions.sh)

This is one of the cleaner parts.

- The pre-deploy safety check for colliding with root symlinks (lib, lib64, bin, sbin) is excellent — same smart guard as in the Python builder and installer.
- tar ... --keep-directory-symlink --no-overwrite-dir is the right combination for merged-/usr systems.
- Manifest generation and archive creation are straightforward and human-readable (Slackware-style).

Potential issues:

- Staging creates symlinks in \$PKG_DEST to mirror root layout — works, but if a package installs a real directory at those paths, the dangerous check catches it.
- pkg_deploy uses a pipe of two tar processes — efficient, but if the first tar fails, the second may still run partially (rare in practice).
- No UID/GID remapping (expected for root-built chroot).
- pkg_manifest uses find . -mindepth 1 — good, but directories get trailing / only if listed that way.

Verdict: Safe enough for trusted packages built in-order. The dangerous check + tar flags provide good protection against the classic merged-/usr foot-gun.

4. Checkpoint Save/Restore

Mechanism: teardown → tar --one-file-system -C "\$IGOS" -czf ... → re-setup.

Pros: Simple, captures the entire state, resume is explicit. **Cons/Risks:**

- Tearing down and re-mounting during a long build can be slow and error-prone.
- If teardown partially fails, the tar may include virtual filesystems or leave the chroot inconsistent.
- No exclusion of /proc, /sys, etc., beyond --one-file-system (which helps but isn't perfect if bind mounts are active).
- Checkpoint directory is hardcoded under /home/\$USER — fine for VM, but document it.
- Restore is manual (rm -rf && tar -xzf) — no automated --restore-from flag.

Reliability: Acceptable for development, but a failure mid-save can leave a half-torn-down chroot. Consider using rsync --delete or btrfs snapshots if the build VM supports it for faster, atomic checkpoints.

5. CWD Reset Between Build Phases

The explicit cd "\$workdir" before each phase (configure, build, check, install, post_install) in build_ch8_package (and similar in other tier scripts) is a **good and necessary** fix. Bash does not scope cd to functions or subshells in the way one might expect.

Is it sufficient? Mostly yes for the current structure. Edge cases remain:

- If a build.sh sources another script that does cd and doesn't return cleanly.

- Background jobs or & processes that outlive the phase.
- The Python igos-build path (desktop tier) doesn't have this issue because it's a separate process.

Recommendation: Keep the resets, and consider adding `cd /` at the very end of each package build for extra hygiene.

6. Package Tracking Correctness

`pkg-functions.sh` is well-designed for its purpose.

- Manifests are human-readable and useful for auditing.
- `pkg_stage` + mirror symlinks + dangerous check = solid.
- Archive creation and deploy are clean.
- `pkg_remove` sorts files in reverse (good for directories).

Minor gaps:

- No content verification on deploy (relies on tar success).
- `pkg_owner` and `pkg_files` scan all manifests linearly — fine for ~500 packages, but could be optimized later with the SQLite DB from `pkm`.
- Config file preservation is missing here (present in the Python `pkm` remover).

Race conditions: Low — builds are sequential and single-threaded per package.

7. Signal Handling

The trap in `build-intergenos.sh` is helpful (prints resume command). However:

- It only runs at the top level; inner scripts (`chroot-enter.sh`, tier builders) may not propagate signals cleanly.
- `set -e` can interfere with trap behavior in some bash versions.
- No trap in most sub-scripts for partial cleanup.

Adequacy: Good enough for interactive use. For production builds, consider a more comprehensive cleanup function that always calls `chroot-teardown.sh`.

Broader Feedback & Recommendations

Strengths:

- Excellent resume support via `--start-at`, `IGOS_START_AT`, and stop file.
- Clear phase logging and timing.
- Hybrid Python + shell approach leverages strengths of both (Python for complex desktop dependency graph; shell for LFS fidelity).
- `create-image.sh` produces a ready-to-boot QCOW2 with sensible VM tweaks (serial console, networking).

Areas for Improvement:

1. **Reduce duplication:** `pkg-functions.sh` and the Python builder/`pkm` have overlapping logic (manifests, staging safety, deploy). Long-term, consider making the Python builder the single source of truth and calling it from all tiers (you already do this for desktop).
2. **Improve teardown robustness:** Add recursive/lazy unmount and verification.
3. **Checkpoint reliability:** Consider excluding more paths or using a faster snapshot method.
4. **Error handling in tier scripts:** Many `run_package` calls exit on first failure — good, but ensure logs clearly point to the failing `build.sh`.
5. **Security / quoting:** Audit remaining unquoted expansions, especially in `create-image.sh` (tar pipes, `chroot` commands).
6. **Testing:** The desktop tier relies on the Python builder — ensure `igos-build` is fully integrated and tested before heavy use.
7. **Documentation:** Add a README explaining the full flow, how checkpoints work, and common resume commands.

Specific quick wins:

- In `chroot-teardown.sh`, replace individual `umount` with a safer loop or `umount -R -l "$IGOS"`.
- In `pkg_deploy`, add a post-deploy verification step (e.g., check a few key files exist).
- Centralize more paths in a single config file sourced by all scripts.

This orchestration layer is mature enough for repeated successful builds in a VM. With the safety improvements to mounts, checkpoints, and quoting, it will be quite reliable. The transition toward the Python builder for more tiers is the right direction — it reduces shell complexity over time.

The whole InterGenOS stack (Python build system → `pkm` → Forge installer → these orchestration scripts) forms a coherent, auditable from-source distribution pipeline. Nice

work keeping it maintainable despite the inherent complexity of building a full distro from scratch.

If you want concrete patches (e.g., improved teardown, safer tar handling, or a unified config), or a deeper dive into any specific script (create-image.sh or a sample build.sh), just say the word!